

Volumes and Content

A Nasdanika Zephyr Book

Pavel Vlasov

Version 2026.11.0, 2026-11-07

Table of Contents

Colophon . . . 1

Preface . . . 2

- Who this book is for . . . 2
- A book that stands on its own . . . 2
- What you need . . . 3
- Conventions . . . 3

I: The Why . . . 4

- **1. Files everywhere** . . . 5
  - 1.1. One abstraction for both directions . . . 5
  - 1.2. A URI is what, a path is where . . . 5
  - 1.3. Agents read files . . . 6
- **2. Why not NIO** . . . 7
  - 2.1. Prior art . . . 7
  - 2.2. Strings, not Path . . . 7
- **3. Content, not streams** . . . 8
  - 3.1. Three rules . . . 8
  - 3.2. Content produced by a writer . . . 8
  - 3.3. The hash . . . 9

II: The How . . . 10

- **4. Getting started** . . . 11
  - 4.1. Adding the module . . . 11
  - 4.2. Three packages . . . 12
- **5. The volume SPI** . . . 13
  - 5.1. Entries . . . 13
  - 5.2. Capabilities . . . 13
- **6. A volume of one's own** . . . 15
  - 6.1. A running JVM as files . . . 17
- **7. Directories and adapters** . . . 18
  - 7.1. A volume over a directory . . . 18
  - 7.2. Adapters . . . 19
- **8. Content** . . . 20
  - 8.1. Factories . . . 20
  - 8.2. Content produced by a writer . . . 21
- **9. Models in volumes** . . . 22

9.1. How URIs map to paths . . . 23
9.2. What the handler does . . . 23
10. Serving over HTTP and WebDAV . . . 24
10.1. The handlers . . . 24
10.2. Serving a volume . . . 25
10.3. Reading over HTTP . . . 25
10.4. One trace across client and server . . . 26
11. Telemetry of transfers . . . 28
III: Looking Ahead . . . 29
12. Looking ahead . . . 30
12.1. More volumes . . . 30
12.2. The file model . . . 30
12.3. More surfaces . . . 30
12.4. A file system that reacts . . . 31
Appendix A: Reference . . . 32
A.1. Java types . . . 32
A.2. Paths . . . 32
A.3. Capabilities . . . 32
Appendix B: Building this book . . . 34
Appendix C: Glossary . . . 35

Colophon

Volumes and Content: A Nasdanika Zephyr Book

Edition 2026.11.0, built on 2026-11-07.

This edition describes the volume module of the Nasdanika SDK Runtime 2026.11.0, Maven coordinates `org.nasdanika.sdk.runtime:volume:2026.11.0`, Java module `org.nasdanika.sdk.runtime.volume`. Every listing in it is included from the module's tests, which are compiled and run when the book is built.

Copyright © 2026 Pavel Vlasov.

The text of this book is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). You may share and adapt it for any purpose, commercial purposes included, provided you give appropriate credit, provide a link to the license, and indicate if you made changes. A suggested credit: "Pavel Vlasov, *Volumes and Content: A Nasdanika Zephyr Book*, licensed under CC BY 4.0", with a link to github.com/Nasdanika-SDK/runtime.

The code is licensed under the Eclipse Public License 2.0, the license of the runtime. In addition, you may use the code of this book's listings in your own programs and documentation without asking for permission and without attribution.

Neither license covers the Nasdanika name or logo: CC BY 4.0 grants no trademark rights.

Java is a trademark of Oracle and its affiliates. Eclipse and EMF are trademarks of the Eclipse Foundation. OpenTelemetry is a trademark of The Linux Foundation. Other names may be trademarks of their owners. Their use here identifies the technologies the module relates to and implies no endorsement.

The source of this book, its tests and the module it describes are at github.com/Nasdanika-SDK/runtime, in `volume`.

Typeset with Asciidoctor PDF in Noto Sans and M+ 1mn, in the look of the Nasdanika Zephyr Books.

Preface

This book is about two small abstractions of the Nasdanika SDK Runtime. A volume is a tree of files and directories behind five methods, wherever the files live: on a disk, in a zip, in memory, in a remote system with a file-shaped API, or nowhere at all, computed from a model or a running JVM. A content is the bytes of a file, as a source that can be read more than once and does nothing until it is read.

Around them, the module has what makes them useful: a volume over any NIO directory, adapters between kinds of content, an EMF URI handler that stores models in volumes, and handlers that serve volumes over HTTP and WebDAV, each request in one trace with its client.

The first part of the book is the why: files as the meeting point of models, people and agents, why the runtime does not implement NIO's file system providers, and why bytes are content and not streams. The second part is the how, for Java developers. The third says what comes next.

Who this book is for

Java developers who store models or generated files somewhere other than a working directory, or who want a model, a data set or a running system to be readable as files: by a browser, a mounted drive, or an agent with file tools.

Writers of backends for remote systems with file-shaped APIs, who want to implement a file system in an afternoon.

A book that stands on its own

This book is one of the Nasdanika Zephyr Books: in-depth manuals, one per module of the Nasdanika SDK Runtime, in the manner of IBM Redbooks. *Harness*, the book of the module that wraps the server of this one for tests, is close to it. You do not need it, or any other book of the series, or the Nasdanika SDK Runtime book, to read this one: where the books touch, this one repeats what you need.

The series is named after the Bootswatch theme it is styled with, Zephyr, and has nothing to do with the Zephyr real-time operating system or with SmartBear's Zephyr test management.

What you need

- **A JDK for Java 25** and **Maven 3.9.16 or later**.
- The module is `org.nasdanika.sdk.runtime.volume`, with the Maven coordinates `org.nasdanika.sdk.runtime:volume:2026.11.0`.
- For Chapter 9, some familiarity with EMF's resources and URIs; for Chapter 10 and Chapter 11, with HTTP and OpenTelemetry.

Conventions

- **Every listing is included from a test** of the module and compiled and run when the book is built. Callouts, such as <1>, explain the lines they mark.
- **Status labels** mark what this edition does not have yet: **AVAILABLE** for what is built, where a table mixes the two, **PLANNED** for designed but not built, **SKETCH** for partly built.
- **Voice.** I write in the first person.

> Editor's notes record inconsistencies between the module and its documentation found while writing. They are shown in builds for the author and left out of published editions.
>
> 
>
> The module had no readme. Its documentation was the chapters "Volumes and content" and "Serving over HTTP and WebDAV" of the Nasdanika SDK Runtime book, whose examples use the book's library model; this book has its own example, a glossary, so that its tests need only the module.

Part I: The Why

Why files are the meeting point of models, people and agents, why the runtime owns a small file system interface instead of implementing NIO's, and why bytes are content and not streams. No code is needed to read this part.

Chapter 1. Files everywhere

Models live in files, and files live in more places than a disk: a zip, a Git repository reached over a REST API, an in-memory sandbox for a test, a remote system with a file-shaped API. Code that loads and saves models should not care which.

The opposite direction is just as useful. A model, a running JVM or an issue tracker presented as files can be read by everything that reads files: a browser, a mounted drive in Finder or Explorer, a Markdown viewer, `grep`, and an agent with file tools. Every tool that understands files then understands the model, at the level of detail the presentation chooses, without knowing that EMF exists.

1.1. One abstraction for both directions

The runtime has one small abstraction for both, the volume:

- **Storing.** A resource set loads and saves models through a volume, so moving models from a working tree to an in-memory volume, an overlay or a remote system changes one handler, not the models and not the references between them.
- **Presenting.** A volume can compute its files from anything: the glossary of this book's examples, the modules of a running JVM, a resource set.
- **Serving.** Anything presented as a volume is served over HTTP to browsers and over WebDAV to anything that mounts drives.

1.2. A URI is what, a path is where

The separation that makes this work is between identity and location. A model's URI is what the model is, and stays the same wherever the model is stored. A path in a volume is where its bytes happen to be. The module's EMF URI handler maps one to the other (Chapter 9), and nothing else needs to know.

1.3. Agents read files

An agent's most dependable tools are file tools: list, read, search, write. A system presented as files is a system an agent can explore without a bespoke integration, with links between files as the relations between elements. A volume over a model is therefore also an interface for agents, and the cheapest one to build.

Chapter 2. Why not NIO

The JDK has a file system abstraction, `java.nio.file`, and it is a pleasure to consume. It is expensive to implement. A `FileSystemProvider` needs a `Path` class with `resolve`, `relativize`, `normalize` and `toUri`, channels, attribute views, a registry of providers keyed by URI scheme, and care wherever the paths of two file systems meet.

The runtime implements file systems often: over models, over Git, over REST APIs, as overlays. So it owns a small interface for implementing them, and uses NIO for consuming: any NIO file system, the default one, a zip or an in-memory one, becomes a volume through one adapter (Chapter 7).

2.1. Prior art

This is where prior art converged:

- **Go's `io/fs`** is almost the same design: an interface of a few methods, string paths with strict rules, and optional interfaces discovered by type assertion.
- **Python's fsspec** has string paths and dozens of backends, from local files to cloud storage.
- **Hadoop's `FileSystem`, Apache Commons VFS and Eclipse EFS** made the same decision in Java: a file system interface of their own, with NIO or `java.io` as one backend among many.

Commons VFS is also the reminder to keep the core small. A volume has five methods; everything else is an optional capability.

2.2. Strings, not Path

A path in a volume is a string, validated once at the boundary, with Go's rules: slash-separated, relative to the root, with no leading or trailing slash and no empty, `.` or `..` segments; the root is `.`. There is no `Path` class to implement, and URIs appear only at the edges, where a volume meets EMF or HTTP.

Chapter 3. Content, not streams

A file's state is a `Content`: a re-readable source of bytes, in the spirit of Guava's `ByteSource` and of the JDK HTTP client's `BodyPublishers.ofInputStream(Supplier)`.

An `InputStream` is one-shot and its ownership is unclear: who closes it, and what if two consumers need the bytes? A content can be read more than once, for a hash, then for a write, then by each subscriber, and it owns nothing until it is opened.

3.1. Three rules

- **Getting content does no I/O.** Opening it does. Asking a volume for the content of a path that does not exist succeeds; opening it fails.
- **Assigning content does not move bytes.** Bytes move when a consumer reads the content: a commit, an HTTP response, a hash. An I/O failure surfaces when the bytes move, not at assignment.
- **Consumers push when they can.** They call `writeTo(out)` for a response, a file, a digest or an upload; `openStream()` only when an input stream is required; and `publisher()` for the JDK HTTP client.

3.2. Content produced by a writer

Much content is produced by code that writes to an output stream: saving a resource, generating a page, encoding text. Turning such code into an input stream is the classic problem of the pipe, and the JDK's `PipedInputStream` is a poor answer: it detects a dead writer or reader from thread liveness, and has a buffer of one kilobyte.

A content produced by a writer runs the writer on the consumer's own stream when the consumer pushes, with no pipe and no thread, as JAX-RS's `StreamingOutput` does. Only a consumer that insists on pulling gets a pipe, a lazy one on a virtual thread. So copying a model from one volume to another serializes it straight into the target's stream, and nothing is serialized if nothing reads it.

3.3. The hash

A content and an entry may know their hash without reading: a content hash, or a Git blob id. It is the most valuable piece of metadata a volume can report. It gives HTTP ETags and conditional requests, suppression of writes that change nothing, incremental copies, and caches of blobs, and it is what lets file systems that react to their own changes converge (Chapter 12).

Part II: The How

The module in Java: the SPI, a volume of one's own, volumes over directories and adapters, content, models stored in volumes, serving over HTTP and WebDAV, and the telemetry of transfers.

Chapter 4. Getting started

4.1. Adding the module

The runtime's bill of materials, `org.nasdanika.sdk.runtime:bom`, manages the versions of the runtime's modules and their dependencies. Import it in `dependencyManagement`, and depend on the module without a version:

```
<dependency>
    <groupId>org.nasdanika.sdk.runtime</groupId>
    <artifactId>volume</artifactId>
</dependency>
```

The module is a named Java module:

Listing 1. The module declaration of the volume module

```
module org.nasdanika.sdk.runtime.volume {

    requires transitive io.opentelemetry.api;
    requires transitive io.opentelemetry.context;
    requires transitive java.net.http;
    requires transitive org.nasdanika.sdk.runtime.common;
    requires transitive org.eclipse.emf.common;
    requires transitive org.eclipse.emf.ecore;
    requires transitive jdk.httpserver;

    exports org.nasdanika.sdk.runtime.volume;
    exports org.nasdanika.sdk.runtime.volume.http;
    exports org.nasdanika.sdk.runtime.volume.emf;

}
```

It requires the runtime's `common` module, for resource sets and telemetry, EMF, the OpenTelemetry API, and two modules of the JDK: the HTTP client, for content at URLs, and the HTTP server, `jdk.httpserver`, for serving volumes. It needs no OpenTelemetry SDK; an application that exports telemetry brings its own.

4.2. Three packages

Package	What it has
`org.nasdanika.sdk.runtime.volume`	`GenericVolume`, `Volume`, their capabilities, and `Content`
`org.nasdanika.sdk.runtime.volume.emf`	`VolumeURIHandler`, which stores EMF resources in a volume
`org.nasdanika.sdk.runtime.volume.http`	`VolumeHttpHandler`, `WebDavHandler`, `TelemetryHandler` and `MediaTypes`, for the JDK's HTTP server

Chapter 5. The volume SPI

`GenericVolume<C>` is the SPI, with the type of content as a parameter; `Volume` is the flavor whose content is bytes, `Content`. A volume has five methods:

Method	Contract
`stat(path)`	The entry at a path, or empty when nothing is there
`list(directory)`	The children of a directory, in a stable order. Blocking, which is fine on virtual threads
`content(path)`	The content at a path, lazily: no I/O happens until it is opened, and opening the content of a missing path throws `NoSuchFileException`
`caseSensitive()`	Declared, because some clients ignore case: Windows Explorer over WebDAV, macOS by default
`capability(type)`	An optional capability, or empty

5.1. Entries

An `Entry` is a record of a path, a kind, `FILE`, `DIRECTORY` or `LINK`, a size, a hash and a modification time. The size is -1 when unknown, and for directories; the hash and the time are null when unknown.

A volume that knows its hashes cheaply should report them (Chapter 3). A volume served over WebDAV should know the sizes of its files, because clients need them to read files reliably.

5.2. Capabilities

Everything beyond reading is optional, and discovered by asking for it:

Capability	Adds	Status
`Writable`	`write(path, content)`, `delete`, `move`, atomic where the backend supports it and `atomicMove()` says so, and `createDirectory`	**AVAILABLE**
`Watchable`	`changes()`, a `Flow.Publisher` of change batches: how a backend reports changes underneath it	**SKETCH**

Capability	Adds	Status
`Transactional`	`begin()`, then `commit(message)` or `rollback()` on the transaction	**SKETCH**
`Searchable`	`glob(pattern)` and `grep(regex, glob)`, pushed down to a backend that can do better than a walk	**SKETCH**
`Linked`	`readLink(path)`: in a model presented as files, references that are not containment are links	**SKETCH**

The interfaces of all five are in the module. "Available" means that a volume of the runtime implements the capability; the others are declared so that the volumes to come, and the code that will use them, agree on their shape.

A change batch has a sequence number, changes and an origin; a change is a `CREATE`, `MODIFY`, `DELETE` or `MOVE` of a path, or `RESYNC`, alone in a batch, which tells a subscriber that fell behind to scan again.

A read-only volume implements the core and answers empty to every capability. A volume over a Git repository will implement the core, `Writable`, `Transactional` and `Watchable`, and nothing forces it to implement the rest.

Chapter 6. A volume of one's own

Here is a complete read-only volume: a glossary presented as Markdown files, an index and a page per term, linked to each other. It is built from a map of terms to definitions, and computes its files once; a volume over a larger source would compute them when asked.

Listing 2. A glossary as files (`GlossaryVolume.java`)

```
/**
 * A glossary presented as Markdown files: an index, and a page per term that links back to
 * it. A browser, a WebDAV client or an agent with file tools reads it like any folder.
 *
 * <p>
 * A read-only volume is the core of the SPI: stat, list, content, case sensitivity, and no
 * capabilities.
 */
public class GlossaryVolume implements Volume {

    private final Map<String, byte[]> files = new TreeMap<>();
    private final Instant modified = Instant.now();

    public GlossaryVolume(Map<String, String> terms) {
        StringBuilder index = new StringBuilder("# Glossary\n\n");
        terms.forEach((term, definition) -> {
            String path = "terms/" + slug(term) + ".md";
            String page = "# " + term + "\n\n" + definition + "\n\n[Glossary](../index.md)\n";
            files.put(path, bytes(page));
            index.append("- [").append(term).append("](").append(path).append(")\n");
        });
        files.put("index.md", bytes(index.toString()));
    }

    @Override
    public Optional<Entry> stat(String path) {
        if (".".equals(path) || "terms".equals(path)) {
            return Optional.of(new Entry(path, Entry.Kind.DIRECTORY, -1, null, modified));
        }
        byte[] bytes = files.get(path);
        if (bytes == null) {
            return Optional.empty();
        }
        Entry file = new Entry(path, Entry.Kind.FILE, bytes.length, hash(bytes), modified);
        return Optional.of(file);
    }

    @Override
    public List<Entry> list(String directory) throws IOException {
        if (".".equals(directory)) {
            return List.of(stat("index.md").orElseThrow(), stat("terms").orElseThrow());
        }
        if (!"terms".equals(directory)) {
```

```
            throw new NoSuchFileException(directory);
        }
        List<Entry> entries = new ArrayList<>();
        for (String path: files.keySet()) {
            if (path.startsWith("terms/")) {
                entries.add(stat(path).orElseThrow());
            }
        }
        return entries;
    }

    @Override
    public Content content(String path) {
        byte[] bytes = files.get(path);
        if (bytes == null) {
            return () -> {
                throw new NoSuchFileException(path); // Lazy: opening fails, asking does not
            };
        }
        return Content.of(bytes);
    }

    @Override
    public boolean caseSensitive() {
        return true;
    }

    @Override
    public <T> Optional<T> capability(Class<T> type) {
        return Optional.empty(); // Not writable, not watchable, not searchable
    }
```

Reading it is reading files:

Listing 3. Reading a volume (`VolumeBookTests.java`)

```
Volume glossary = new GlossaryVolume(GlossaryVolume.terms());
assertThat(glossary.list("."))
    .extracting(Volume.Entry::path)
    .containsExactly("index.md", "terms");
assertThat(glossary.stat("terms/content.md")).hasValueSatisfying(entry -> {
    assertThat(entry.kind()).isEqualTo(Volume.Entry.Kind.FILE);
    assertThat(entry.hash()).isNotNull();                                    ①
});
try (InputStream in = glossary.content("terms/content.md").openStream()) {
    assertThat(new String(in.readAllBytes(), StandardCharsets.UTF_8))
        .startsWith("# Content")
        .contains("[Glossary](../index.md)");
}
Content missing = glossary.content("terms/missing.md");                      ②
assertThatThrownBy(missing::openStream).isInstanceOf(NoSuchFileException.class);
```

① The hash is known without reading the file. The HTTP handler turns it into an ETag.

② Asking for the content of a missing file succeeds. Opening it fails.

A Markdown viewer renders the glossary as a small site, and an agent with file tools navigates it by following links. Chapter 10 serves the same volume to browsers and to WebDAV clients.

6.1. A running JVM as files

The module's tests have a larger example, `JvmModuleVolume`: the modules of a module layer, one file per module, grouped into directories by the segments of their names as a Maven repository groups artifacts, so that `java.base` is `java/base.html`. Each page lists what the module requires, exports and provides, with links to the pages of the modules it requires. It renders HTML for browsers, or Markdown for agents reading a mounted share, and computes each page on first use, so that sizes and hashes are known.

`JvmVolumeServer`, a main class in the tests, serves the running JVM's own modules both ways, for a browser and for a WebDAV mount.

Chapter 7. Directories and adapters

7.1. A volume over a directory

`Volume.of(path)` is a writable volume over a directory of any NIO file system: the default one, a zip file system, an in-memory one.

- It validates paths, so that they cannot escape the root.
- It probes the file system for case sensitivity, instead of guessing it from the operating system.
- It keeps the semantics of a file system: writing into a missing directory fails, and so does deleting a directory that is not empty.
- A write that fails leaves the target as it was: content goes to a temporary file beside the target, and is moved over it when complete.

Listing 4. A volume over a directory (`VolumeBookTests.java`)

```
Volume volume = Volume.of(directory);
Volume.Writable writable = volume.capability(Volume.Writable.class).orElseThrow();
Content hello = Content.of("# Hello", StandardCharsets.UTF_8);

assertThatThrownBy(() -> writable.write("notes/hello.md", hello))
    .isInstanceOf(NoSuchFileException.class);                          ①
writable.createDirectory("notes");
writable.write("notes/hello.md", hello);
writable.move("notes/hello.md", "notes/welcome.md");                   ②

assertThat(volume.list("notes"))
    .extracting(Volume.Entry::path)
    .containsExactly("notes/welcome.md");
assertThat(directory.resolve("notes/welcome.md")).hasContent("# Hello");
assertThatThrownBy(() -> volume.stat("../outside"))
    .isInstanceOf(IllegalArgumentException.class);                     ③
```

① The directory does not exist yet. A volume does not create directories behind its user's back; the EMF handler of Chapter 9 creates them on purpose.

② Atomic on file systems that support it.

③ Paths are validated at the boundary.

7.2. Adapters

`adapt(readMapper, writeMapper)` turns a volume of one kind of content into a volume of another. It is how the runtime's TypeScript generator writes text, without converting it to bytes itself:

Listing 5. A text volume over a directory (`VolumeBookTests.java`)

```
GenericVolume<String> text = Volume.of(directory).adapt(
        content -> {                                                    ①
            try (InputStream in = content.openStream()) {
                return new String(in.readAllBytes(), StandardCharsets.UTF_8);
            } catch (IOException e) {
                throw new UncheckedIOException(e);
            }
        },
        string -> Content.of(string, StandardCharsets.UTF_8));          ②

@SuppressWarnings("unchecked")
GenericVolume.Writable<String> writable = text.capability(GenericVolume.Writable.class)
        .orElseThrow();
writable.write("hello.md", "# Hello");
assertThat(text.content("hello.md")).isEqualTo("# Hello");
assertThat(directory.resolve("hello.md")).hasContent("# Hello");
```

① Reading maps bytes to text. The mapping runs when content is asked for, so a text volume reads eagerly.

② Writing maps text to bytes, encoded when the target volume reads the content.

`Volume.of(genericVolume)` goes the other way, and views a `GenericVolume<Content>` as a `Volume`.

Chapter 8. Content

`Content` has one abstract method, `openStream()`, so a lambda is a content, as the glossary's missing files are. Its default methods give the rest: `writeTo(out)`, which pulls and copies unless the content can push; `length()` and `hash()`, empty unless known; `repeatable()`; and `publisher()`, a `Flow.Publisher` of byte buffers for the JDK HTTP client's `BodyPublishers.fromPublisher`.

8.1. Factories

Factory	Content
`Content.of(bytes)`	In memory, copied, with a known length
`Content.of(text, charset)`	Text, encoded when read
`Content.of(supplier)`	From a stream supplier, called on every open
`Content.of(uri)`	At a URI, requested on every open: HTTP and HTTPS through the JDK HTTP client, other schemes through `URL`. A 404 is a `NoSuchFileException`
`Content.once(stream)`	A stream someone handed over. Not repeatable
`Content.ofWriter(writer)`	Produced by a closure that writes to an output stream, when the content is read
`Content.empty()`	No bytes

Listing 6. Content (`VolumeBookTests.java`)

```
Content bytes = Content.of("in memory".getBytes(StandardCharsets.UTF_8));
assertThat(bytes.length()).hasValue(9);                                  ①

AtomicInteger runs = new AtomicInteger();
Content written = Content.ofWriter(out -> {
    runs.incrementAndGet();
    out.write("written on demand".getBytes(StandardCharsets.UTF_8));
});
assertThat(runs).hasValue(0);                                            ②
ByteArrayOutputStream out = new ByteArrayOutputStream();
written.writeTo(out);                                                    ③
assertThat(read(written)).isEqualTo(out.toString(StandardCharsets.UTF_8)); ④
assertThat(runs).hasValue(2);

Content once = Content.once(InputStream.nullInputStream());
assertThat(once.repeatable()).isFalse();                                 ⑤
```

① In memory: the length is known without reading.

② Creating content produced by a writer runs nothing.

③ Pushing runs the writer on the consumer's stream.

④ Pulling runs it again, through a pipe. Each read is a run.

⑤ A consumer that needs to read a one-shot content twice buffers it first.

8.2. Content produced by a writer

`Content.ofWriter` turns a closure that writes to an output stream into content without running it:

- `writeTo(out)` runs the writer on the consumer's stream, with no pipe and no thread.
- `openStream()` returns a lazy pipe. The writer starts on a virtual thread at the first read, and writes bounded chunks into a queue that the reader drains. A failure of the writer is rethrown to the reader, and a reader that closes early cancels the writer.
- Each read runs the writer again, so the content is repeatable as long as the writer is. `ofWriter(writer, false)` declares a writer that is not.

Copying a model to another volume takes three lines, and the model is serialized straight into the target's stream:

Listing 7. A model as content (`Models.java`)

```
/**
 * Copies a model as content produced by a writer: nothing is serialized until the target
 * volume reads the content, and it is serialized straight into the target's stream.
 */
static void copy(Resource resource, Volume target, String path) throws IOException {
    Content content = Content.ofWriter(out -> resource.save(out, null));      ①
    Volume.Writable writable = target.capability(Volume.Writable.class)       ②
            .orElseThrow(() -> new IOException("Read-only volume"));
    writable.write(path, content);                                             ③
}
```

① Nothing is serialized here. The content is a promise to save the resource when it is read.

② Writing is a capability: a read-only volume refuses it by not having it.

③ The volume reads the content, which runs the save on the volume's own stream.

Chapter 9. Models in volumes

`VolumeURIHandler` bridges EMF onto a volume: URIs below a base URI are paths in the volume, so a resource set loads and saves resources through the volume's reads and writes. The model of this chapter is an Ecore package with a class per term of the glossary, so that the book needs no model of its own; any EMF model is stored the same way.

Listing 8. A resource set over a volume (`Models.java`)

```
static final URI BASE = URI.createURI("glossary://models/");

/**
 * A resource set whose glossary:// URIs are paths in the volume.
 */
static ResourceSet resourceSet(Volume volume) {
    ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure();
    URIConverter converter = resourceSet.getURIConverter();
    converter.getURIHandlers().add(0, new VolumeURIHandler(volume, BASE));    ①
    TelemetryURIHandler.wrapAll(converter, () -> Telemetry.of(resourceSet)); ②
    return resourceSet;
}

static void save(EPackage model, Volume volume, String path) throws IOException {
    URI uri = BASE.appendSegments(path.split("/"));                           ③
    Resource resource = resourceSet(volume).createResource(uri);
    resource.getContents().add(model);
    resource.save(null);                                                      ④
}

static EPackage load(Volume volume, String path) {
    URI uri = BASE.appendSegments(path.split("/"));
    Resource resource = resourceSet(volume).getResource(uri, true);
    return (EPackage) resource.getContents().getFirst();
}
```

① URIs below `glossary://models/` are paths in the volume. The handler goes first, before EMF's default handlers.

② A resource set configured by the runtime has every handler instrumented, but this one was added later, so it is wrapped too. Reads and writes become `URIHandler.read` and `URIHandler.write` spans.

③ `glossary://models/2026/glossary.ecore` is the path `2026/glossary.ecore`.

④ The bytes are buffered and written as one content when the stream is closed, so a failed save writes nothing. Missing parent directories are created first.

Listing 9. Saving and loading through a volume (`VolumeBookTests.java`)

```
Volume volume = Volume.of(directory);                                    ①
Models.save(Models.glossary(), volume, "2026/glossary.ecore");           ②
assertThat(directory.resolve("2026").resolve("glossary.ecore")).exists(); ③
EPackage loaded = Models.load(volume, "2026/glossary.ecore");
assertThat(loaded.getEClassifiers()).hasSize(4);
```

① A writable volume over a temporary directory.

② The resource's URI is `glossary://models/2026/glossary.ecore`, whatever the directory is.

③ The handler created the missing directory `2026` before writing.

9.1. How URIs map to paths

- The URI `<base>a/b%20c.xmi` is the path `a/b c.xmi`: the part below the base, split into segments, each decoded. The base itself is the root.
- A query and a fragment are ignored, so the URI of an object maps to the path of its resource.
- URIs whose decoded path has empty, `.` or `..` segments, or an encoded slash, are not handled.
- A base without a trailing separator gets one, so `glossary://models` and `glossary://models/` are the same base.

9.2. What the handler does

- **Reading** opens the path's content. A missing path is a `NoSuchFileException`.
- **Writing** needs the `Writable` capability, and is otherwise refused as read-only.
- **Attributes** come from `stat`: the time stamp, the length, whether a path is a directory, whether it is read-only, which it is without the `Writable` capability, and the hash, as the attribute `hash`. Setting attributes is ignored, as by EMF's default handler.

The model never mentions where it is stored. Moving models from a working tree to an in-memory volume, a Git overlay, or a volume over a remote system changes one handler, not the models and not the references between them.

Chapter 10. Serving over HTTP and WebDAV

Anything presented as a volume can be served: to a browser over HTTP, and to anything that mounts drives over WebDAV. A model becomes a folder in Finder or Explorer, a preview of a generated site is one URL away, and an agent reads a running system with the file tools it already has.

The module serves volumes with the HTTP server the JDK has shipped for years, `com.sun.net.httpserver` in the module `jdk.httpserver`. With virtual threads it is enough for local tools, previews and agents, with no servlet container and no framework.

10.1. The handlers

`VolumeHttpHandler` serves a volume read-only, in the manner of the JDK's `SimpleFileServer`:

- Files are sent with a media type from their extension, a `Last-Modified` header from the entry, and an ETag from the entry's hash. A conditional request whose `If-None-Match` matches gets a `304`.
- Directories are generated HTML listings. A directory requested without a trailing slash is redirected to it, so relative links in pages and listings resolve.
- The request path below the context path is the volume path. Paths with empty, `.` or `..` segments are refused.
- `GET` and `HEAD` are served; other methods get a `405`.

`WebDavHandler` extends it into a read-only WebDAV (RFC 4918) share of class 1:

- `OPTIONS` answers `DAV: 1` and the allowed methods.
- `PROPFIND` answers a `207 Multi-Status` with the resource type, display name, content length, content type, ETag and modification time, as far as the volume knows them. `Depth: infinity` is answered as `1`, so that a client cannot make the server walk a large volume.
- `GET` and `HEAD` work as in `VolumeHttpHandler`, listings included, so the share also browses in a web browser.
- Methods that write or lock get a `405`.

That is enough to mount a share in macOS Finder ("Connect to Server"), on Windows (`net use`), on Linux (davfs2) or with rclone, and to read it with any tool.

10.2. Serving a volume

Listing 10. Serving a volume (`Serve.java`)

```
public static HttpServer serve(Volume volume, OpenTelemetry openTelemetry, int port)
        throws IOException {
    var address = new InetSocketAddress(InetAddress.getLoopbackAddress(), port);
    HttpServer server = HttpServer.create(address, 0);
    server.setExecutor(Executors.newVirtualThreadPerTaskExecutor());              ①

    HttpHandler files = new VolumeHttpHandler(volume);
    server.createContext("/glossary", new TelemetryHandler(files, openTelemetry)); ②

    HttpHandler share = new WebDavHandler(volume);
    server.createContext("/dav", new TelemetryHandler(share, openTelemetry));     ③

    server.start();
    return server;
}
```

① A virtual thread per request. Handlers block, and blocking is cheap.

② Browsers read the glossary at `/glossary/`. Each request is a unit of work with a server span.

③ WebDAV clients mount the same volume at `/dav/`.

The server binds to the loopback address. The handlers do not authenticate, so serving beyond the local machine needs what the JDK server provides for it, an `HttpsServer` and an `Authenticator` on the context, or a proxy in front. `Serve`, a main class in the tests, prints the two URLs and serves until Enter is pressed.

10.3. Reading over HTTP

On the client side, content at a URL is a `Content` like any other:

Listing 11. Content at a URL (`VolumeBookTests.java`)

```
Content index = Content.of(URI.create(root + "/glossary/index.md")); ①
assertThat(read(index)).startsWith("# Glossary");
```

① Nothing is requested until the content is opened, and every open requests it again.

A WebDAV listing is a `PROPFIND` request, which the JDK HTTP client sends like any other method:

Listing 12. Listing a directory over WebDAV (`VolumeBookTests.java`)

```
HttpResponse<String> listing = HttpClient.newHttpClient().send(
        HttpRequest.newBuilder(URI.create(root + "/dav/terms/"))
            .method("PROPFIND", HttpRequest.BodyPublishers.noBody())
            .header("Depth", "1")
            .build(),
        HttpResponse.BodyHandlers.ofString());
assertThat(listing.statusCode()).isEqualTo(207);                    ①
assertThat(listing.body()).contains("volume.md");
```

① Multi-Status: a response per member of the directory.

10.4. One trace across client and server

`TelemetryHandler` wraps a handler, or `TelemetryHandler.filter(openTelemetry)` is added as a filter of a context, and makes each request a unit of work:

- The trace context is extracted from the request headers with the propagators of the OpenTelemetry instance, so the client's span is the parent of the server's.
- The server span is named `<method> <route>`, the route being the context path, and carries the attributes of the HTTP semantic conventions: method, route, path, scheme, server and client address, protocol version, user agent, response status and response body size.
- The instance and the span are current while the wrapped handler runs, so whatever it does, content transfers, resource loads, its own log records, reports to the request.
- A failure of the handler is recorded on the span. If no response was sent yet, the client gets a `500`; if one was, the connection is dropped instead of completing a response that would look successful.
- A failure or a `5xx` sets the error status. A `4xx` is the client's error, not the server's, and leaves the status unset.

On the client side, `Content.of(uri)` starts a client span and propagates its context in the request headers. Together they make one trace:

Listing 13. A trace that crosses HTTP (`VolumeBookTests.java`)

```
InMemorySpanExporter exporter = InMemorySpanExporter.create();
var tracerProvider = SdkTracerProvider.builder()
```

```
        .addSpanProcessor(SimpleSpanProcessor.create(exporter))
        .build();
try (OpenTelemetrySdk sdk = OpenTelemetrySdk.builder()
        .setPropagators(Telemetry.w3cPropagators())                    ①
        .setTracerProvider(tracerProvider)
        .build()) {
    HttpServer server = Serve.serve(volume, sdk, 0);                   ②
    int port = server.getAddress().getPort();
    URI index = URI.create("http://127.0.0.1:" + port + "/glossary/index.md");
    try (Scope scope = Telemetry.makeCurrent(sdk);                     ③
            InputStream in = Content.of(index).openStream()) {
        in.readAllBytes();
    } finally {
        server.stop(0);
    }
    SpanData client = await(exporter, SpanKind.CLIENT);
    SpanData handled = await(exporter, SpanKind.SERVER);
    assertThat(handled.getTraceId()).isEqualTo(client.getTraceId());   ④
    assertThat(handled.getParentSpanId()).isEqualTo(client.getSpanId());
    assertThat(handled.getName()).isEqualTo("GET /glossary");
}
```

① W3C trace context propagation, on both sides: the client injects, the server extracts.

② The server's instance. In a real deployment it is another process with its own instance; here both share one, so the test can see both spans.

③ The client's unit of work. Opening the content is a `GET` client span.

④ The server span is a child of the client span, in the same trace.

In a test, the setup above is the same every time: a server, an executor, a filter on every context, a client that propagates. The runtime's `harness` module packages it in one method, `Harness.inServer`, and its Zephyr Book, *Harness*, covers it.

Chapter 11. Telemetry of transfers

Moving bytes is work, and it is reported to the OpenTelemetry instance of the unit of work, the one that the runtime's `Telemetry.current()` returns:

Span	When
`Content.writeTo`	Content pushed to a stream, with the byte count, unless the content is in memory
`Content.pipe`	Content produced by a writer and read as a stream: from the first read to the end of the writer. The writer runs in it, so the spans the writer starts are its children
`GET` (client)	Content at an HTTP URI, from the request to the close of the body, with the attributes of the HTTP semantic conventions and the trace context in the request headers
`Content.read`	Content at another URI, from opening to closing
`URIHandler.read`, `URIHandler.write`	A stream opened through an instrumented URI handler, from opening to closing
`<method> <route>` (server)	A request handled through `TelemetryHandler` (Chapter 10)

All of them emit progress log records at most once a second while bytes move, so a long download shows in a live view of the telemetry while it happens.

Crossing to the writer's virtual thread carries the OpenTelemetry instance and the parent span along, so the spans of a writer that saves a resource are children of the transfer that caused them.

The URL recorded on a client span has no user info and no query string, which may carry credentials.

With no OpenTelemetry instance current, the spans go to the no-op instance and cost nothing, so code that moves bytes is instrumented once and runs the same in a unit of work that records and one that does not.

Part III: Looking Ahead

More volumes, the file model on top of them, more surfaces, and file systems that react.

Chapter 12. Looking ahead

The volume is the bottom layer of a larger design.

12.1. More volumes

PLANNED

- **An in-memory volume**, for tests and sandboxes. The module's tests have one, `MemoryVolume`, which may become public.
- **An overlay**, which keeps writes in memory over a read-only base until commit.
- **A Git volume**, whose commit is a Git commit, locally or over the GitHub and GitLab APIs without a clone.
- **A model volume**, which presents a resource set as folders, files and links.
- **Volumes over systems**: the JVM, then trackers such as Azure DevOps, with caching and polling.

12.2. The file model

SKETCH

Files and directories as an Xcore model over volumes, in the runtime's `models/file`: `File`, `Directory`, `TextFile` and types specialized by content type, with operations such as `view`, `replace` and `insert` that are also an agent's file tools. Views decode content on get and encode it lazily on set, so two views of one file cannot diverge. The model and its generated code are in the runtime; its behavior is being attached.

12.3. More surfaces

PLANNED

- **Writable WebDAV**, behind the `Writable` capability, so that editing a mounted model is editing the model.
- **VS Code**, through its `FileSystemProvider` API, which consumes the change batches of a `Watchable` volume directly.

- **FUSE**, for a native mount on Linux and macOS.

12.4. A file system that reacts

PLANNED

Processors, deterministic or agentic, react to file changes and change files in turn until nothing changes, and then one commit records the run. The hash in every entry is what lets such loops converge: a write that changes nothing is suppressed, and so is the reaction to it.

Appendix A: Reference

A.1. Java types

Type	What it is for
`GenericVolume<C>`	The SPI, with the type of content as a parameter, its `Entry`, its capabilities, and `adapt`
`Volume`	A volume of bytes. `Volume.of(path)` is a writable volume over a directory; `Volume.of(genericVolume)` views a volume of `Content` as a `Volume`
`Volume.Writable`	The `Writable` capability for bytes
`Content`	A re-readable source of bytes, and its factories
`emf.VolumeURIHandler`	Stores EMF resources in a volume, below a base URI
`http.VolumeHttpHandler`	Serves a volume read-only over HTTP
`http.WebDavHandler`	Serves a volume as a read-only WebDAV share
`http.TelemetryHandler`	Makes each request a unit of work with a server span; `filter(openTelemetry)` as a context filter
`http.MediaTypes`	Media types from file extensions

A.2. Paths

Path	Valid
`.`	The root
`terms/volume.md`	Yes
`/terms`, `terms/`	No: no leading or trailing slash
`a//b`, `./a`, `a/../b`	No: no empty, `.` or `..` segments

A.3. Capabilities

Capability	Methods
`Writable`	`write(path, content)`, `delete(path)`, `move(from, to)`, `createDirectory(path)`, `atomicMove()`, `adapt(mapper)`
`Watchable`	`changes()`: a `Flow.Publisher<ChangeBatch>`
`Transactional`	`begin()`: a `Transaction` with `commit(message)`, `rollback()` and `close()`, which rolls back unless committed
`Searchable`	`glob(pattern)`, `grep(regex, glob)`: matches with a path, a line and its text
`Linked`	`readLink(path)`

Appendix B: Building this book

This book is AsciiDoc in `volume/src/docs/asciidoc`, built with the module after its tests have passed:

```
mvn -pl volume -am verify                          # the module, its tests and this book
mvn -pl volume verify -Dzephyr.editor-notes=false  # without editor's notes
mvn -pl volume verify -Dasciidoctor.skip=true      # without the book
```

The PDF and the single-page HTML are written to `target/book` and attached to the module's artifact with the classifier `book`; the test sources are attached with the classifier `test-sources`. The look comes from the runtime's `doc/zephyr` directory, shared by all the Nasdanika Zephyr Books.

Listings are tagged regions of the tests in `src/test/java/org/nasdanika/sdk/runtime/volume/book`, and of the module declaration. A listing line holds at most 96 characters, and a warning fails the build.

The tests also have two main classes to try the module by hand: `book.Serve`, which serves the glossary for a browser and a WebDAV mount, and `http.JvmVolumeServer`, which serves the running JVM's modules the same way.

Appendix C: Glossary

Capability

An optional ability of a volume, such as writing, found by asking for it.

Content

A re-readable source of bytes: getting it does no I/O, and nothing moves until it is read.

Entry

What a volume knows about a path without reading it: its kind, size, hash and modification time.

ETag

An HTTP header that identifies a version of a resource, here the entry's hash, so that a client can ask for a resource only if it changed.

Path

A slash-separated string relative to the root of a volume, with Go's rules; the root is `.`.

Unit of work

What a span and its children cover: here, a request handled by the server, or a transfer of content.

Volume

A tree of files and directories behind five methods, wherever the files live, or computed from something else.

WebDAV

An extension of HTTP, RFC 4918, for reading and writing files over HTTP, which operating systems mount as drives.